Nätverksautomation

Typer av SDN

- **Device-based SDN**
  - Programmerbara enheter, t.ex. Python-libraries för att kommunicera och automatisera konfiguration på enheter, utan någon controller
- **Controller-based SDN**
  - Centraliserad controller med kunskap om hela nätverket
  - Regler definieras med kod
- **Policy-based SDN**
  - Än enklare än *controller-based* då controllern kan styras med ett enkelt webbgränssnitt där man sätter policies (ingen programmeringskunskap krävs)
  - T.ex. APIC-EM

Nätverksautomation

- Nätverksautomation är nära besläktat med **Device-Based SDN**
- Automation är en process som sköter sig själv, och som minskar eller tar bort behovet av mänsklig interaktion
- Att konfigurera och uppdatera enstaka enheter är ofta inget problem, men tänk om du är ansvarig för en miljö med hundratals enheter
- Med nätverksautomatisering kan vi utföra effektivare underhåll och med säkrad kvalitet

Dataformat

- För att stödja automation måste enheter ha ett standardiserat sätt att kommunicera med varandra och med människor
- När vi kommunicerar med enheterna måste vi använda ett gemensamt dataformat som är enkelt för enheten att förstå
- HTML är ett väldigt enkelt dataformat som används för hemsidor
- **Livelabb:** HTML

Dataformat

- Anger regler för hur data kan presenteras:
  - Syntax, t.ex. hakparentes [], parenteser (), måsvingar {}, kommatecken ,
  - Datarepresentation, t.ex. strängar, listor, arrayer
  - Nycklar och värden
- Ett vanligt sätt att presentera data är med **JSON (JavaScript Object Notation)**:


```
{"message": "success", "timestamp": 1560789216,
"iss_position": {"latitude": "25.9990",
"longitude": "-132.6992"}}
```

Korrekt formatterad JSON

```
{
        "message": "success",
        "timestamp": 1560789260,
        "iss_position": {
                "latitude": "25.9990",
                "longitude": "-132.6992"
        }
}
```

YAML (YAML Ain’t Markup Language)

```yaml
message: success
timestamp: 1560789260
iss_position:
    latitude: '25.9990'
    longitude: '-132.6992'
```

XML (eXtensible Markup Language)

```xml
<?xml version="1.0" encoding="UTF-8" ?>
<root>
  <message>success</message>
  <timestamp>1560789260</timestamp>
  <iss_position>
    <latitude>25.9990</latitude>
    <longitude>-132.6992</longitude>
  </iss_position>
</root>
```

JSON

- Fördelen att använda ett standardiserat dataformat som JSON är att det är enklare för maskiner att tolka
- Jämför IOS-output:

```
GigabitEthernet0/0/0 is up, line protocol is up (connected)
  Description: Wide Area Network
  Internet address is 172.16.0.2/24
```

- Med JSON-output:

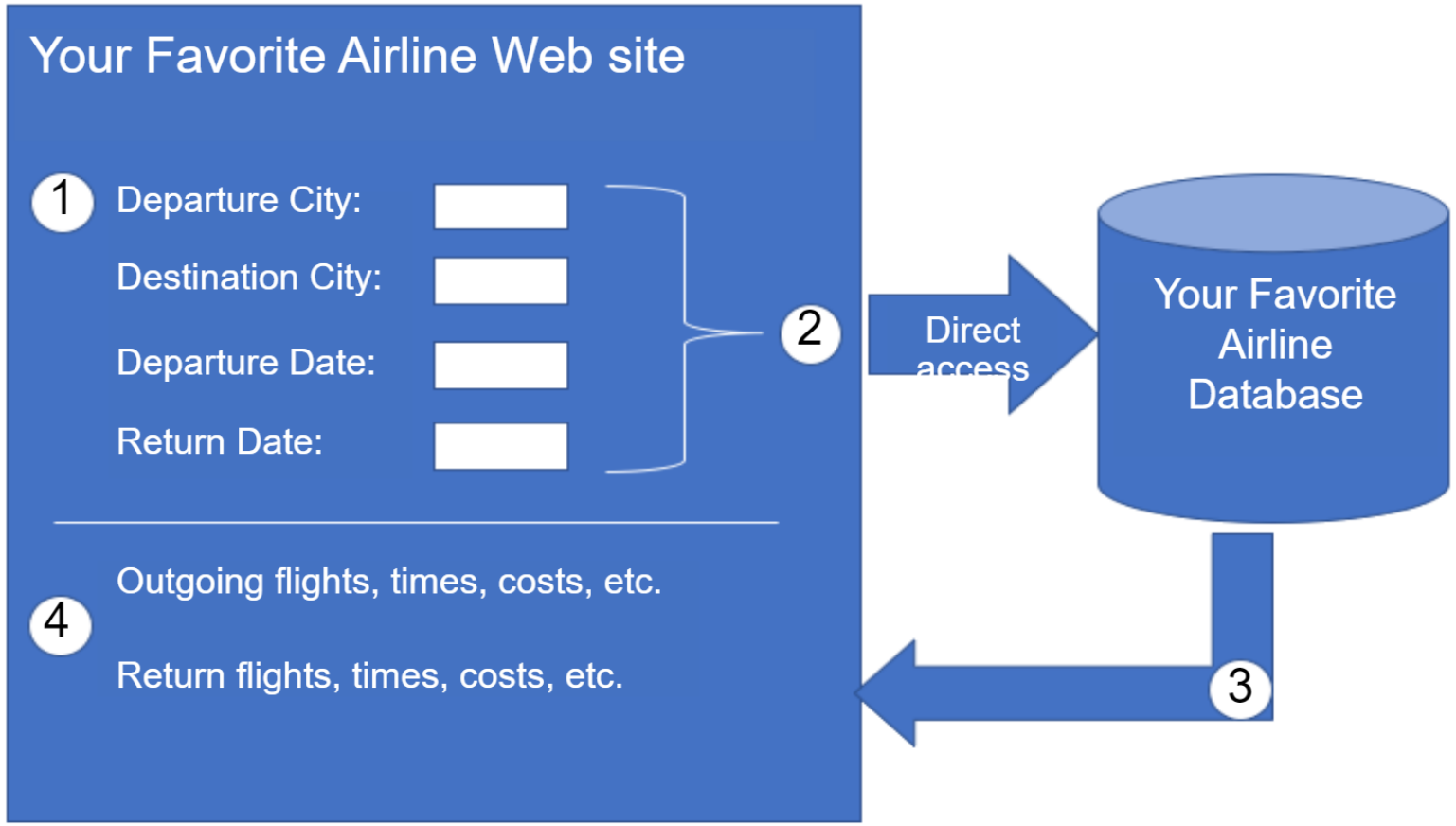
```
{
    "ietf-interfaces:interface": {
        "name": "GigabitEthernet0/0/0",
        "description": "Wide Area Network",
        "enabled": true,
        "ietf-ip:ipv4": {
            "address": [
                {
                    "ip": "172.16.0.2",
                    "netmask": "255.255.255.0"
                }
            ]
        }
    }
}
```

API (Application Programming Interface)

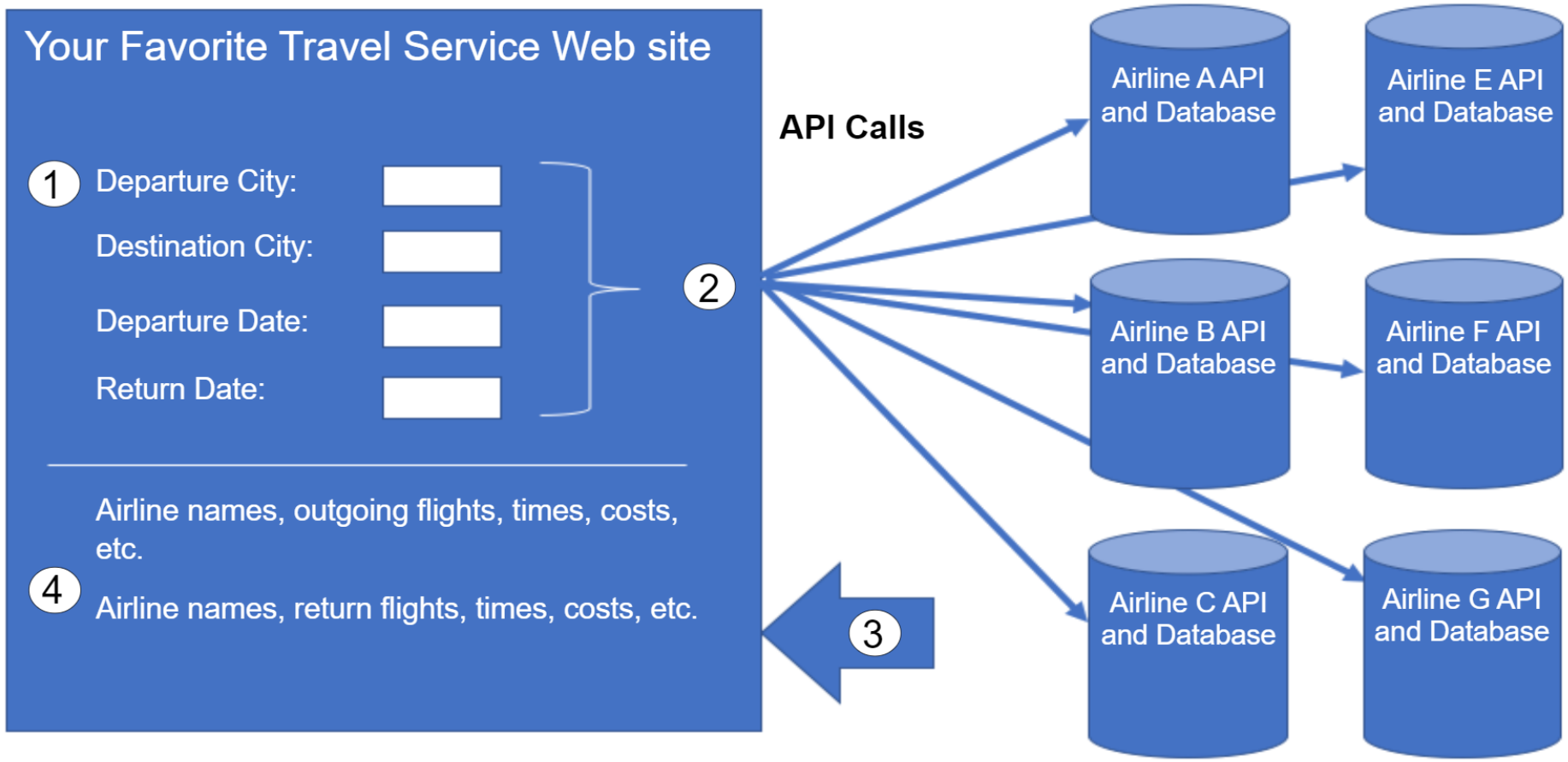
- Ett API låter applikationer kommunicera med varandra enligt ett standardiserat dataformat
- Ett API är också ett sätt för klienter att kommunicera mot applikationer
- Svaret på en API-förfrågan ser ut som följande:

```
{
        "message": "success",
        "timestamp": 1560789260,
        "iss_position": {
                "latitude": "25.9990",
                "longitude": "-132.6992"
        }
}
```

Exempel på API



Exempel på API



Tre olika API

- **Open/Public APIs**
  - Tillgängliga för alla, t.ex. positionen för NASA ISS
  - Antingen helt öppna, eller via en gratis token
- **Internal/Private APIs**
  - Används inom en organisation
  - Ibland via Internet, men i så fall via en låst token
- **Partner APIs**
  - Används mot partners, t.ex. i fallet med resebyrå som har API mot flygbolag

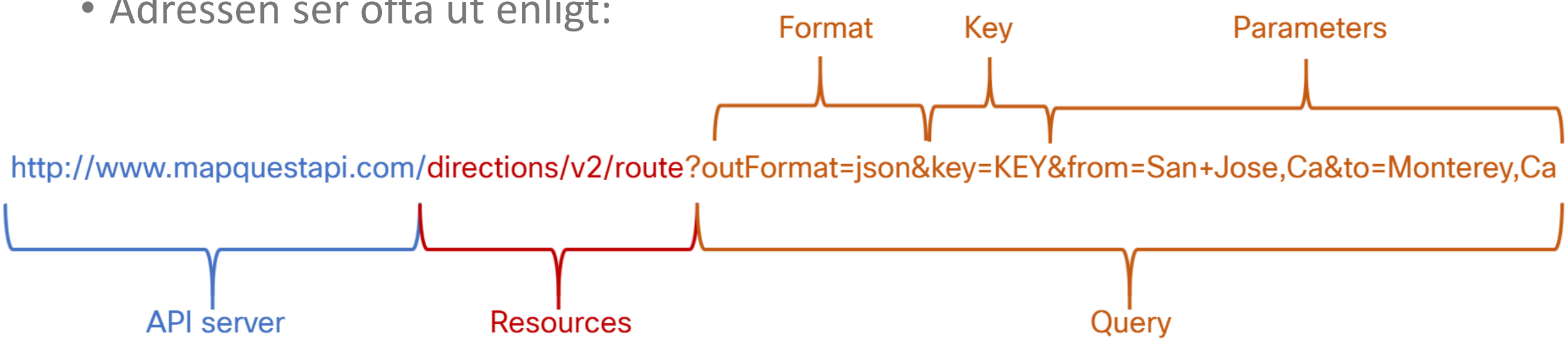
Web Service API

- En majoritet av alla API bygger på såkallade ”Web Services”, som är egentligen innebär att de bygger på HTTP

Characteristic	SOAP	REST	XML-RPC	JSON-RPC
Data Format	XML	JSON, XML, YAML, and others	XML	JSON
First released	1998	2000	1998	2005
Strengths	Well-established	Flexible formatting and most widely used	Well-established, simplicity	Simplicity

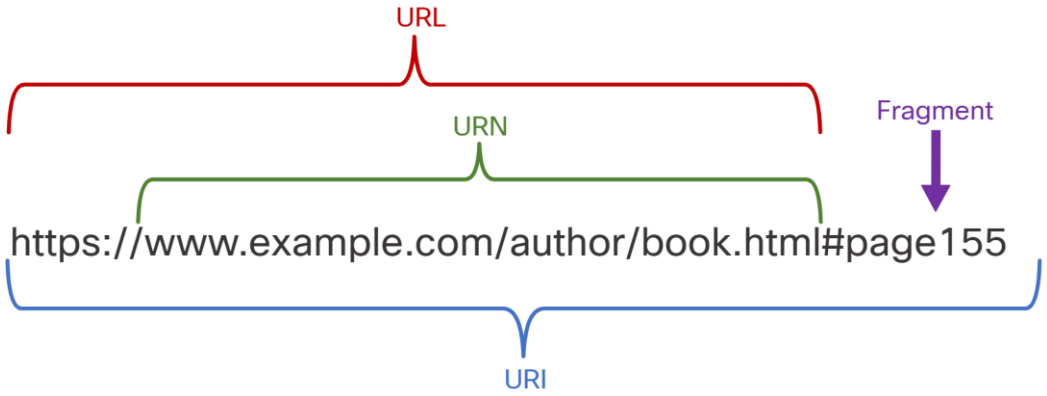
REST och RESTful API

- Precis som att en webbläsare använder HTTP GET för att hämta information på en webbserver, använder REST sig av HTTP GET för att hämta information på en tjänst
- Adressen ser ofta ut enligt:



URI, URN & URL

- Eftersom REST API kommunicerar med webbadresser är det viktigt att hålla isär begreppen:
  - **Domännamn** (www.example.com)
  - **URI** (Uniform Resource Identifier), hela adressen
  - **URN** (Uniform Resource Name), hela adressen utan protokoll och fragment
  - **URL** (Uniform Resource Locator), endast ”namespace” inklusive resurs



REST-anrop

```
GET /people HTTP/1.1
Host: www.namefinder.com
Content-Type: text/html; charset=utf-8
Content-Length: nnn
```

Hämtar tekniskt sett endast informationen på URI/URL

http://www.namefinder.com/people

REST-svar

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: nnn

{
  "first name": "Alice",
  "last name": "Smith",
  "age": 25,
  "address": {
    "street address": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postal code": "10021"
  }
}
```

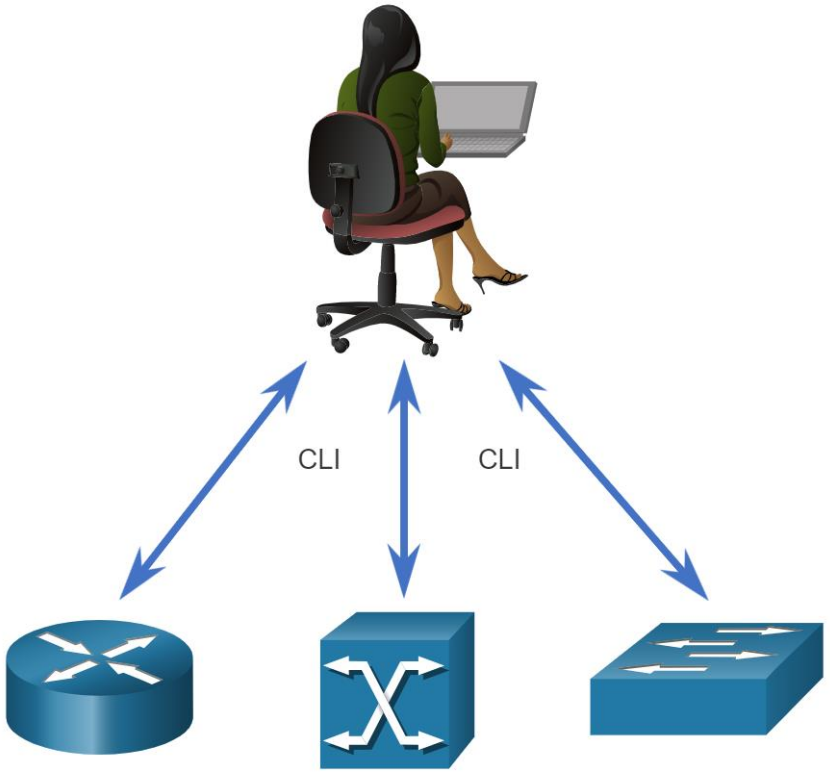
http://ip.jsontest.com/

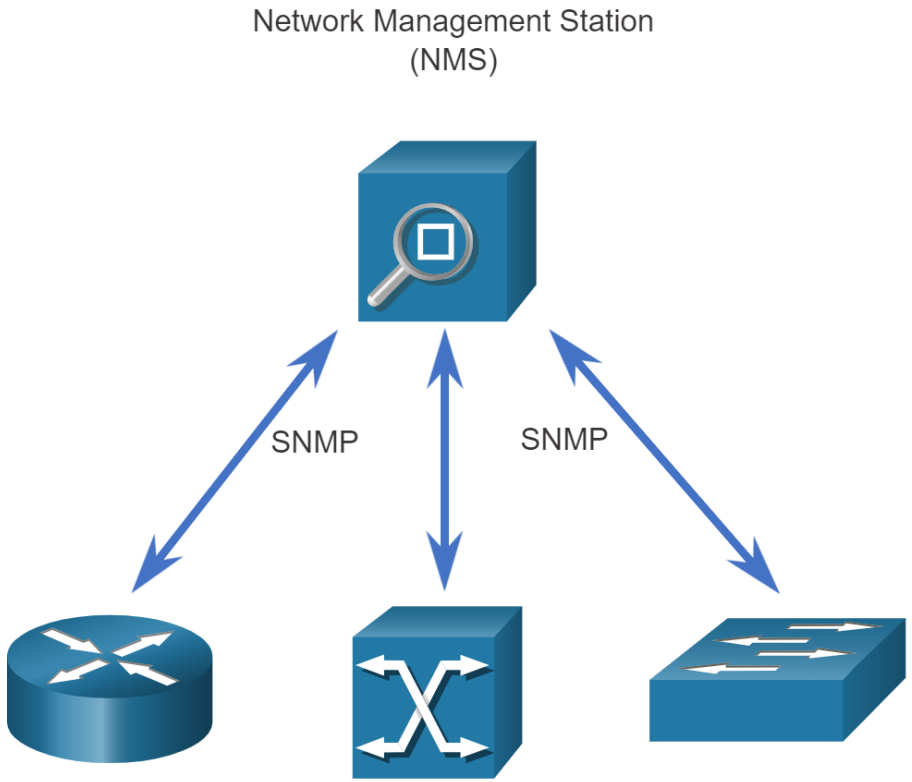
Hur används APIer?

- Många tjänster har utvecklarsidor (**Exempel:** developer.spotify.com)
- Postman (**livelabb**)
- Python-skript
- Nätverksoperativsystem använder sig av NETCONF och RESTCONF för att förenkla automation

- Utnyttjas också av verktyg för **Configuration Management**

Configuration Management (hur ni gör nu)





Configuration Management

- Består av flera **Configuration Items (CIs)**
  - Vilken konfiguration har enheter, tjänster och system?
  - Förr i tiden Excel-dokument med dokumentation, nu för tiden:
- CM-verktyg som automatiserar uppgifter via REST APIs, det kan handla om:
  - Kontroll på mjukvara och versioner
  - Enhetsattribut, namn, adresser, säkerhet
  - ACL
- Kallas ibland även **Infrastructure as Code (IaC)**

Ansible

- **Ansible** är det mest använda verktyget för Configuration Management
  - Öppen källkod (ägs av Red Hat)
  - Har brett stöd bland olika plattformar, men körs från Linux/Unix
  - Kör agentlöst, via t.ex. SSH och PowerShell
- Använder **playbooks** för att automatisera konfiguration och driftsättning av enheter

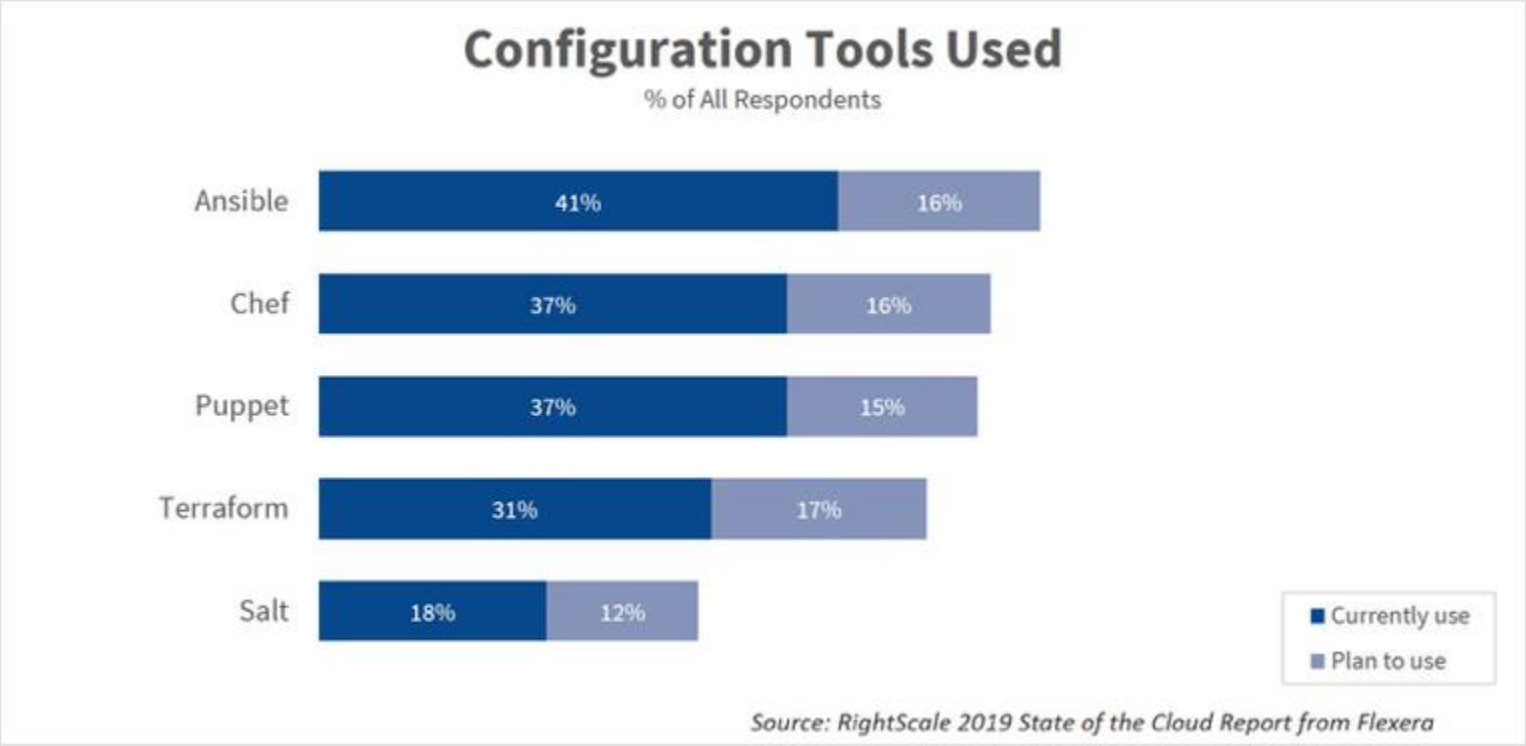
Ansible playbook (YAML)

```yaml
---
- hosts: all
  gather_facts: no


  tasks:

  - name: SET COMMUNITY STRING
    ios_config:
      lines:
        - snmp-server community dd19 RO 1

- name: ENABLE OSPF TRAPS
    ios_config:
      lines:
        - snmp-server enable traps ospf state-change
        - snmp-server enable traps ospf errors

- name: SET SNMP SERVER ADDRESS
    ios_config:
      lines:
        - snmp-server host 10.0.0.4 version 2c public udp-port 161
```

Jämförelse mellan verktyg

Characteristic	Ansible	Chef	Puppet	SaltStack
What programming language?	Python + YAML	Ruby	Ruby	Python
Agent-based or agentless?	Agentless	Agent-based	Supports both	Supports both
How are devices managed?	Any device can be “controller”	Chef Master	Puppet Master	Salt Master
What is created by the tool?	Playbook	Cookbook	Manifest	Pillar

Popularitet



IBN & DNA Center

- Väldigt Cisco-specifikt… Kanske utdaterad när ni slutar, läs på inför certifikatstest
- Cisco har ganska nyligen släppt ett nytt certifikat: **Cisco DevNet Associate** (DEVASC 200-901)
- Kommer mer i senare kurser!

JÖNKÖPING UNIVERSITY

School of Engineering